Package 'fastphylosig'

October 27, 2026

Type Package

Title Fast Phylogenetic Signal Statistics

Version 0.3.1

Description Provides one-command and method-specific workflows for phylogenetic signal in continuous, binary, and categorical traits. Compiled tree-pruning algorithms implement Blomberg's K and Pagel's lambda without constructing a dense phylogenetic covariance matrix in production. Fritz and Purvis D retains separate random and Brownian null models, while categorical Delta combines discrete ancestral-state likelihoods with MCMC and permutation diagnostics. The package also provides reusable prepared trees, explicit tree/data matching, trait-wise missing-value handling, conservative tree diagnostics, bounded simulation memory, and structured timing and Monte Carlo uncertainty metadata. Methods follow Blomberg, Garland and Ives (2003) <doi:10.1111/j.0014-3820.2003.tb00285.x> for K, Pagel (1999) <doi:10.1038/44766> for lambda, Fritz and Purvis (2010) <doi:10.1111/j.1523-1739.2010.01455.x> for D, and Borges et al. (2019) <doi:10.1093/bioinformatics/bty800> for categorical Delta.

License MIT + file LICENSE

Encoding UTF-8

Depends R (>= 4.1.0)

Imports ape, parallel, Rcpp, stats

LinkingTo Rcpp, RcppArmadillo

Suggests caper, mvtnorm, phytools, testthat

URL https://github.com/yinwen-ecology/fastphylosig, https://yinwen-ecology.github.io/fastphylosig/

BugReports https://github.com/yinwen-ecology/fastphylosig/issues

SystemRequirements C++17, OpenMP (optional)

Config/testthat/edition 3

NeedsCompilation yes

Author Yin Wen [aut, cre]

Maintainer Yin Wen <wenyinsci@126.com>

Repository CRAN

Date/Publication 2026-10-27 14:40:02 UTC

Contents

fastphylosig-package 2
cache_info 3
check_tree 4
fast_ace 5
fast_d 6
fast_delta 8
fast_k 10
fast_lambda 11
fast_signal 13
match_phylo_data 14
match_tree_data 15
plot_signal 16
prepare_tree 17
resolve_tree 18
result_methods 19

Index **21**

`fastphylosig-package` *Fast phylogenetic signal calculations*

Description

One-command and method-specific workflows for selected phylogenetic signal statistics. The method-explicit `fast_signal(tree, data, method, ...)` entry point checks and matches the inputs, then runs K, lambda, D, or Delta. Continuous-trait Blomberg's K and Pagel's lambda preserve the `phytools::phylosig()` targets for `se = NULL`; the binary Fritz and Purvis D statistic follows `caper::phylo.d()`; categorical Delta uses accelerated discrete ancestral-state likelihoods and reports MCMC diagnostics.

Details

Start with `fast_signal` for a standard analysis. Advanced users can run tree and data preparation explicitly, then call `fast_k`, `fast_lambda`, `fast_d`, `fast_delta`, and `fast_ace`. Repeated analyses can reuse `prepare_tree`. Utilities include `cache_info`, `check_tree`, `resolve_tree`, `match_tree_data`, and `plot_signal`.

Production K and lambda calculations use compiled tree-pruning algorithms. K randomizations are evaluated in bounded chunks, and lambda optimization is completed within one compiled batch call. Public utilities provide method-specific tree checks, conservative representation normalization, progress output, runtime reporting, and structured results without exposing numerical-engine choices.

Missing values are handled trait by trait. Retained-tip patterns are grouped, and species matching/removal counts are reported. D retains separate random and Brownian null P values. K, D, and Delta report Monte Carlo precision where applicable; Delta additionally reports ESS, split R-hat, and estimate MCSE.

`plot_signal()` provides statistic-specific diagnostics: a permutation null plot for K, a profile-likelihood plot for lambda, a two-null Fritz and Purvis D calibration plot, and a permutation null plot for Delta.

`cache_info` *Inspect a prepared-tree cache*

Description

Reports structural and numerical cache use without modifying the prepared tree. Dense numerical entries are managed by a byte-budget least-recently-used cache.

Usage

```
cache_info(ctx)
```

Arguments

`ctx`	An object returned by `prepare_tree()`.

Value

A list containing `budget`, `bytes_used`, numbers of structural and numerical entries, hit/miss/eviction counters, and an `entries` data.frame. The table reports retained tip count, structural, numerical and total bytes, VCV, Cholesky and eigensystem availability, and last-access order for each key.

Examples

```
set.seed(1)
tree <- ape::rtree(20)
ctx <- prepare_tree(tree, cache_budget = 64 * 1024^2)
cache_info(ctx)
```

check_tree *Check a phylogenetic tree before signal calculations*

Description

Inspect a phylogenetic tree without reordering, pruning, compiling, or otherwise changing it. Readiness is reported separately for each selected signal method. K, lambda, and D can use compatibility paths for polytomies; Delta follows `fast_ace()` and requires a rooted, fully-dichotomous tree with strictly positive branch lengths. The summary reports both the conventional degree-two rooted flag and the structural root used by the compatibility kernels, so a root polytomy or an unrooted encoding is visible rather than silently treated as a binary rooted tree.

Usage

```
check_tree(tree, signal = c("K", "lambda", "D", "Delta"))
```

Arguments

`tree`	A `phylo` object or an object returned by `prepare_tree()`.
`signal`	Character vector selecting one or more methods. The default checks `"K"`, `"lambda"`, `"D"`, and `"Delta"`.

Value

An object of class `fastphylosig_tree_check`. Its `ready` field is a logical scalar and `ready_by_signal` is a named logical vector. `tree_summary` contains tip, topology, rootedness, ultrametricity, branch-length, polytomy, unary-node, and zero/near-zero branch diagnostics. Zero internal branches are conditional for lambda; zero terminal branches require user review at the lambda upper boundary. The `issues` field is a data frame with `code`, `severity`, `signal`, `message`, `check`, `problem`, `action`, and `auto_fixable` columns. Severity is one of `INFO`, `WARNING`, `ERROR`, or `USER_ACTION_REQUIRED`. Representation-only canonical node-ID diagnostics may set `auto_fixable = TRUE`; biological rooting, topology, and branch-length repairs are never automatic. The input tree is never modified.

Examples

```
if (requireNamespace("ape", quietly = TRUE)) {
  tree <- ape::read.tree(text = "((a:1,b:1):1,(c:1,d:1):1);")
  check_tree(tree)
  check_tree(tree, signal = c("K", "lambda"))
}
```

`fast_ace` *Fast discrete ancestral-state reconstruction*

Description

Optimized subset of `ape::ace()` for discrete maximum-likelihood ancestral-state reconstruction. The statistical target follows `ape::ace(type = "discrete", method = "ML")` for `"ER"` and `"ARD"` models, while the repeated pruning likelihood calculation is implemented in Rcpp/RcppArmadillo.

Usage

```
fast_ace(x, phy = NULL, type = "discrete", method = "ML", CI = TRUE,
  model = c("ER", "ARD"), kappa = 1, ip = 0.1, marginal = FALSE,
  prepared = NULL, progress = interactive())
```

Arguments

`x`	Categorical trait vector with one value per tip. Names, when present, must be unique and have exactly the same species set as `phy$tip.label`; their order may differ and values are reordered to tree-tip order.
`phy`	A rooted, fully dichotomous `"phylo"` object with finite, strictly positive branch lengths.
`type`	Currently only `"discrete"` is supported.
`method`	Currently only `"ML"` is supported.
`CI`	Whether to return ancestral likelihoods in `lik.anc`.
`model`	Discrete transition model, `"ER"` or `"ARD"`; the default is `"ER"`.
`kappa`	Optional branch-length exponent, matching `ape::ace()`.
`ip`	Initial value for transition-rate optimization.
`marginal`	Whether to return marginal likelihoods. The default `FALSE` matches the default path used by `ape::ace()`.
`prepared`	An optional object returned by `prepare_tree()`. When supplied, it supplies a validated phylogenetic tree and takes precedence over `phy`. V2 integrity is checked before cached evidence is reused. The ACE likelihood itself uses the tree edges and does not require the dense covariance cache.
`progress`	Whether to report method-level progress stages and elapsed runtime. Defaults to `interactive()`. Setting this to `FALSE` suppresses progress messages; warnings and errors are still signalled.

Value

An `ace`-like object containing `loglik`, `rates`, numerical rate standard errors in `se`, `index.matrix`, optimizer status, and, when `CI = TRUE`, `lik.anc`. A `timing` attribute and list field report elapsed runtime in seconds.

See Also

fast_delta(), fast_signal()

Examples

```
set.seed(5)
tree <- ape::rtree(30)
x <- ape::rTraitDisc(tree, model = "ER", k = 3, rate = 1,
                     states = letters[1:3])
fast_ace(x, tree, model = "ER")
```

---

fast_d *Fast caper-compatible phylo.d calculation*

Description

Computes the Fritz and Purvis D statistic for binary traits using the same null-model structure as caper::phylo.d(). The contrast-sum calculation is moved into C++ for repeated simulation work. Brownian null states are simulated directly along tree edges and then thresholded by the observed state proportion, avoiding dense VCV construction for this step. Binary labels are recoded to equally spaced 0/1 states, making D invariant to the original labels. For ordinary binary trees without controlled state matrices, null generation, thresholding, traversal, and accumulation are fused in chunks. Peak null-state memory is therefore O(n times chunk_size) rather than O(n times nsim). Controlled and polytomy cases use the compatibility path.

Usage

```
fast_d(tree, x = NULL, test = TRUE, nsim = 1000, se = NULL,
  return_sim = test, verbose = TRUE, rnd.bias = NULL, permutations = NULL,
  random_states = NULL, brownian_states = NULL, ncores = 1, X = NULL,
  permut = NULL, prepared = NULL, chunk_size = 128,
  keep_null = return_sim, progress = interactive())
```

Arguments

tree	An object of class "phylo" with finite, strictly positive branch lengths and no single-child internal nodes. Safe representation-only differences in root numbering and edge order are normalized automatically. Ordinary binary trees and rooted polytomies are supported; ultrametricity is not required, while unsafe topology changes remain errors.
x, X	Named binary trait vector or matrix/data.frame of binary traits. X is a compatibility alias for x; supply only one. Species identifiers are matched by name and reordered to tree-tip order.
test	Whether to return randomization and Brownian P values.
nsim, permut	Number of simulations. permut is a caper::phylo.d()-style compatibility alias for nsim; when supplied, it takes precedence.

se	Unused; binary states should be supplied directly in x.
return_sim	Whether random and Brownian simulation vectors may be returned. Its default is test; actual retention also requires keep_null = TRUE.
verbose	Whether to report species matching.
progress	Whether to report method-level progress stages and elapsed runtime. Defaults to interactive(). Setting this to FALSE suppresses progress and species-matching messages; warnings and errors are still signalled.
rnd.bias	Optional sampling weights for the random null model.
permutations	Optional permutation matrix for the random null model.
random_states, brownian_states	Optional state matrices for controlled debugging.
ncores	Number of OpenMP threads for the C++ contrast-sum simulation kernel. Binary trees can use multiple threads, although scaling is often modest because traversal is memory-bandwidth bound. Polytomies fall back to the serial vectorized path to preserve caper-compatible handling.
prepared	An optional object returned by prepare_tree(). When supplied, its V2 integrity is validated before cached tree calculations are reused and tree is ignored.
chunk_size	Integer from 32 to 512 controlling the number of null states generated and traversed per streaming block.
keep_null	Whether full random and Brownian null-statistic vectors may be retained. Both return_sim = TRUE and keep_null = TRUE are required. Set either to FALSE to keep only online means, tail counts, D, and P values.

Value

For vector input, a compact phylo.d-like object without caper's NodalVals or embedded comparative.data. In addition to the legacy Pval1/Pval0 fields, it reports the null-specific aliases P_random/P_Brownian, their Bernoulli Monte Carlo standard errors MCSE_P_random/MCSE_P_Brownian, and nsim_requested, nsim_successful_random, nsim_successful_brownian, nsim_failed_random, and nsim_failed_brownian. The two P values are never combined. For matrix/data.frame input, a data.frame with one row per binary trait and the same fields (with the legacy Pval1_fast/Pval0_fast columns retained for compatibility).

Rooted polytomies use the serial compatibility path. Unary internal nodes are not supported, including unary nodes exposed after species matching or trait-wise missing-value pruning.

Every trait also has a machine-readable status ("ok", "partial", "invalid_trait", "insufficient_data", or "failed") and a human-readable message/note. These fields distinguish invalid traits and incomplete simulation sets from a valid result.

The MCSE uses sqrt(p * (1 - p) / nsim_successful) separately for each null. Thus the reported MCSE is exactly zero when the estimated P is 0 or 1, and is NA when no finite simulation succeeded. Non-finite simulation statistics are excluded from that null's denominator and explicitly reported in the successful/failed counts and note; they are not silently counted as non-extreme values. With test = FALSE, P and MCSE columns are returned as NA, although simulation counts still describe the null calibration used to compute D. Results expose a timing attribute with at least total_elapsed (seconds); vector results also provide a timing list field.

See Also

fast_signal(), check_tree()

Examples

```
set.seed(3)
tree <- ape::rtree(30)
z <- ape::rTraitCont(tree)
x <- as.integer(z > stats::median(z))
names(x) <- tree$tip.label
fast_d(tree, x, nsim = 20, return_sim = FALSE, verbose = FALSE)
```

---

fast_delta *Fast Delta statistic for categorical traits*

Description

Computes the single-tree Delta statistic for categorical traits. Ancestral state probabilities are estimated with fast_ace(), node uncertainty is transformed to entropy, and Delta is the posterior mean(beta) / mean(alpha) from two chains. Ancestral reconstruction, entropy, and MCMC calculations use Rcpp/RcppArmadillo. The ape comparison path is retained internally for regression tests, not exposed as a user engine.

Usage

```
fast_delta(tree, x = NULL, test = FALSE, nsim = 1000, se = NULL,
  mcmc_sim = 10000, thin = 10, burn = 100, lambda0 = 0.1,
  proposal_sd = 0.5, entropy = c("LSE", "SE", "GINI"),
  model = "ARD", permutations = NULL, return_sim = test,
  verbose = TRUE, ncores = 1, X = NULL, prepared = NULL,
  progress = interactive())
```

Arguments

tree	A conventionally rooted, fully binary "phylo" object with finite, strictly positive branch lengths, or a compatible context returned by prepare_tree(). The method does not require ultrametricity.
x, X	Named categorical trait vector or matrix/data.frame. Matrix and data.frame row names must be species names. X is a compatibility alias for x; supply only one. Species identifiers are matched by name and reordered to tree-tip order.
test	Whether to compute a permutation P value.
nsim	Number of trait permutations when test = TRUE.
se	Unused; retained so unsupported measurement-error input fails clearly. Use proposal_sd for the MCMC proposal.
mcmc_sim	Number of MCMC iterations per chain and Delta calculation.

thin	MCMC thinning interval.
burn	First MCMC iteration eligible to be saved.
lambda0	Rate of the exponential priors on alpha and beta.
proposal_sd	Standard deviation of the log-normal random-walk proposal.
entropy	Entropy transformation: "LSE", normalized Shannon entropy "SE", or normalized Gini impurity "GINI".
model	Discrete transition model used by fast_ace(): "ER" or "ARD".
permutations	Optional integer permutation matrix with exactly nsim rows and one column per retained tip.
return_sim	Whether to retain simulated Delta values.
verbose	Whether to report species matching.
progress	Whether to report method-level progress stages and elapsed runtime. Defaults to interactive(). Setting this to FALSE suppresses progress and species-matching messages; warnings and errors are still signalled.
ncores	Number of worker processes for the Delta permutation test. Small jobs can be slower in parallel because process startup is not free. Replaying stochastic results requires a fixed worker count; changing it repartitions worker RNG streams and is not draw-for-draw reproducible.
prepared	Optional object returned by prepare_tree(); its V2 integrity is validated before cached evidence is reused.

Value

For vector input, a phylo_delta-compatible object; matrix/data.frame input returns one row per trait. Results report Delta, alpha/beta chain means and SDs, ESS, split R-hat, covariance-aware MCSE_Delta, requested and successful iteration counts, and safe RNG metadata. Tests also report P, MCSE_P, failed fits, and successful permutation counts. Inspect the diagnostics before interpreting short runs. Results expose a timing attribute with at least total_elapsed (seconds); vector results also provide a timing list field. Per-trait status and message/note fields distinguish successful, partial, invalid-trait, insufficient-data, and failed calculations.

Fewer than two species shared globally by the tree and data is a call-level error. After matching and trait-wise NA pruning, Delta returns a per-trait insufficient-data result when a trait retains fewer than two species. A batch with one or more traits retaining 2–19 species emits one warning and still returns estimates. The metadata records all per-trait retained/removed counts and sample-size status (including the fewer-than-two error), plus the affected trait details, in attr(result, "analysis_metadata")$delta_retained_species.

For a finite Delta estimate, non-finite chain diagnostics, ESS below 20, or split R-hat above 1.1 emit one batch warning. This warning never reruns a chain or alters requested iterations. Per-trait diagnostic warning details are retained in attr(result, "analysis_metadata")$delta_mcmc_diagnostics.

The public defaults include test = FALSE, nsim = 1000, return_sim = test, mcmc_sim = 10000, thin = 10, burn = 100, and model = "ARD". Thus permutation P, MCSE_P, and retained null values are absent unless testing is enabled; null values additionally require return_sim = TRUE. MCSE_P uses the finite successful permutation count. These are computational defaults rather than a guarantee of adequate mixing for every data set. Users should increase the chain when ESS, split R-hat, or Monte Carlo error is inadequate.

See Also

fast_signal(), fast_ace()

Examples

```
set.seed(4)
tree <- ape::rtree(25)
x <- sample(letters[1:3], ape::Ntip(tree), replace = TRUE)
names(x) <- tree$tip.label
fast_delta(tree, x, mcmc_sim = 10000, thin = 10, burn = 100,
           verbose = FALSE)
```

---

fast_k *Fast Blomberg's K phylogenetic signal*

Description

Computes Blomberg's K using the validated tree-pruning production engine. This is the fixed-method specialist counterpart of fast_signal(..., method = "K"). The result class and shape are kept compatible with the former continuous fast_signal() API. Matching, trait-wise missing-value handling, and tree preparation follow the same validated path as the unified dispatcher.

Usage

```
fast_k(tree, x = NULL, test = FALSE, nsim = 1000, se = NULL,
  permutations = NULL, return_sim = FALSE, verbose = TRUE, ncores = 1,
  X = NULL, prepared = NULL, trait_chunk = 64, simulation_chunk = 128,
  keep_null = return_sim, progress = interactive(), data = NULL)
```

Arguments

tree	A connected "phylo" object with a unique structural root and finite, strictly positive branch lengths, or a context returned by prepare_tree(). A conventional degree-two root, binary topology, and ultrametricity are not required.
x, X, data	Named continuous trait vector or trait matrix/data.frame. The data spelling is an alias for x; supply only one spelling.
test	Whether to compute the permutation significance test.
nsim	Number of randomizations when test = TRUE.
se	Currently only NULL is supported.
permutations	Optional integer permutation matrix with exactly nsim rows and one column per retained tip.
return_sim	Whether to return simulated K values.
verbose	Whether to report species matching.
ncores	Number of OpenMP threads for randomization.

prepared	Optional object returned by prepare_tree(); when supplied it is used in place of tree after V2 integrity validation.
trait_chunk	Positive integer number of traits processed per C++ block.
simulation_chunk	Positive integer number of randomizations processed per streaming block.
keep_null	Whether to retain the full K null distribution.
progress	Whether to report method-level progress and elapsed runtime.

Value

For vector input, a phylosig-compatible result (an atomic numeric when test = FALSE); matrix/data.frame input returns one row per trait. Results include timing metadata and a small workflow attribute. Per-trait status and message/note fields identify invalid, insufficient, or undefined traits; an atomic no-test result exposes the same information as attributes.

See Also

fast_signal(), fast_lambda()

Examples

```
tree <- ape::rtree(20)
x <- stats::rnorm(20)
names(x) <- tree$tip.label
fast_k(tree, x, verbose = FALSE)
```

---

fast_lambda *Fast Pagel's lambda phylogenetic signal*

Description

Computes Pagel's lambda using the validated tree-pruning production engine. This is the fixed-method specialist counterpart of fast_signal(..., method = "lambda"). The result class and shape are kept compatible with the former continuous fast_signal() API. When a profile is requested, the grid runs from zero to the tree-specific allowed maximum. The nominal 95 percent profile interval uses max(logL) - 0.5 * qchisq(0.95, df = 1) as its cutoff; bounds are found by linear interpolation at threshold crossings and fall back to grid endpoints when a crossing is absent. The interval is therefore an approximate grid/profile interval, not an exact confidence interval.

Usage

```
fast_lambda(tree, x = NULL, test = FALSE, se = NULL,
  verbose = TRUE, ncores = 1, X = NULL,
  lambda_profile = NULL, lambda_profile_points = 101,
  prepared = NULL, trait_chunk = 64, progress = interactive(), data = NULL)
```

Arguments

tree	A connected "phylo" object with a unique structural root and finite, non-negative branch lengths, or a context returned by prepare_tree(). Zero terminal branches are not supported; zero internal branches are allowed with a numerical warning. A conventional degree-two root, binary topology, and ultrametricity are not required.
x, X, data	Named continuous trait vector or trait matrix/data.frame. The data spelling is an alias for x; supply only one spelling.
test	Whether to compute the likelihood-ratio test against lambda zero.
se	Currently only NULL is supported.
verbose	Whether to report species matching.
ncores	Accepted for API compatibility. The current fixed-lambda and optimization kernels run serially; increasing this value does not enable parallel lambda evaluation.
lambda_profile	Whether to store a lambda likelihood profile. The default stores it for vector input and skips it for batch input.
lambda_profile_points	Number of regular profile grid points (at least 5). The fitted lambda, and lambda 1 when allowed by the tree, are inserted in addition to the regular grid.
prepared	Optional object returned by prepare_tree(); when supplied it is used in place of tree after V2 integrity validation.
trait_chunk	Positive integer number of traits processed per C++ block.
progress	Whether to report method-level progress and elapsed runtime.

Value

For vector input, a phylosig-compatible result; matrix/data.frame input returns one row per trait. Results include timing metadata and a small workflow attribute. Per-trait status and message/note fields distinguish valid fits from invalid, insufficient, or undefined trait-level results.

See Also

fast_signal(), fast_k()

Examples

```
tree <- ape::rtree(20)
x <- stats::rnorm(20)
names(x) <- tree$tip.label
fast_lambda(tree, x, verbose = FALSE)
```

`fast_signal` *Unified fast phylogenetic signal dispatcher*

Description

Provides the one-command entry point for phylogenetic signal analysis and dispatches to exactly one of the four public production functions: `fast_k()`, `fast_lambda()`, `fast_d()`, or `fast_delta()`. The selected function owns validation, progress output, runtime timing, numerical calculation, and result construction. Method aliases are normalized to `"K"`, `"lambda"`, `"D"`, or `"Delta"`. Ordinary calls perform tree inspection, conservative representation handling, explicit species matching, and trait-wise missing- value grouping before the selected production kernel runs. No biological tree repair is inferred from the method or trait type.

Usage

```
fast_signal(tree, data = NULL, method = NULL, ..., x = NULL, X = NULL,
  prepared = NULL, trait_chunk = 64)
```

Arguments

`tree`	A `"phylo"` object or a context returned by `prepare_tree()`.
`x`, `data`, `X`	Named trait vector or trait matrix/data.frame. The `data` spelling is accepted by the unified dispatcher; `x` and `X` are retained as compatibility aliases. Matrix and data.frame row names must be species names. Supply only one spelling.
`method`	Exactly one method selector. Supported aliases are `K`/`k`, `lambda`/`Lambda`, the Unicode `U+03BB` alias, `D`/`d`, and `Delta`/`delta`. The method is explicit and is never inferred from the trait type; omitting it or supplying an unknown selector is an error.
`prepared`, `trait_chunk`	Compatibility formals for prepared tree contexts and C++ trait blocks; prepared contexts undergo V2 integrity validation, and the formals are forwarded only to targets that support them.
`...`	Arguments for the selected production function. K and lambda accept the arguments documented by `fast_k()` and `fast_lambda()`; D and Delta accept those documented by `fast_d()` and `fast_delta()`, respectively. Unknown or inapplicable arguments are reported as errors. Legacy positional arguments after `method` remain supported. When `data`, `x`, and `X` are supplied more than once, the call fails rather than choosing one silently.

Details

For a first analysis, supply a tree, named trait data, and one method selector to this function. Advanced users can call `fast_k()`, `fast_lambda()`, `fast_d()`, or `fast_delta()` directly when they need method-specific simulation, profile, null-retention, MCMC, or diagnostic controls. `fast_ace()` is the specialist ancestral-state ML function used by the Delta workflow.

Value

The original result class and shape returned by the selected production function (for example `phylosig`, `phylo.d`, or `phylo_delta`). A small `workflow` attribute records the canonical method and production function. Lambda results may include an approximate profile and nominal 95 percent interval: the profile uses a regular grid from zero to the tree-specific allowed maximum, with the fitted value (and one when allowed) inserted. Its cutoff is `max(logL) - 0.5 * qchisq(0.95, df = 1)`; limits are estimated by linear interpolation at profile threshold crossings, or by grid endpoints when no crossing is available. This is a grid/profile approximation rather than an exact confidence interval.

See Also

`fast_k()`, `fast_lambda()`, `fast_d()`, `fast_delta()`, `fast_ace()`, `check_tree()`, `match_tree_data()`, `prepare_tree()`

Examples

```
set.seed(2)
tree <- ape::rtree(30)
X <- matrix(stats::rnorm(30 * 3), 30, 3,
            dimnames = list(tree$tip.label, paste0("trait_", 1:3)))
fit <- fast_signal(tree, data = X, method = "K", test = FALSE,
                   verbose = FALSE)
summary(fit)
fast_signal(tree, data = X[, 1], method = "lambda", verbose = FALSE)
```

---

match_phylo_data *Deprecated alias for* `match_tree_data`

Description

This compatibility alias is deprecated. Use `match_tree_data()` instead. The function returns the same object as `match_tree_data()` and emits a deprecation warning when called.

Usage

```
match_phylo_data(tree, X, prune = TRUE, verbose = TRUE)
```

Arguments

`tree`	An object of class `"phylo"` or a prepared tree context.
`X`	Named vector, matrix, or data.frame. Species names must be supplied as names or row names unless rows are already in `tree$tip.label` order.
`prune`	Whether to drop tree tips that are absent from `X`.
`verbose`	Whether to print a short matching message.

Value

See match_tree_data.

Examples

```
set.seed(6)
tree <- ape::rtree(10)
X <- data.frame(trait = stats::rnorm(10), row.names = tree$tip.label)
# This compatibility alias is deprecated; prefer match_tree_data().
matched <- suppressWarnings(match_phylo_data(tree, X, verbose = FALSE))
matched$report
```

---

match_tree_data *Match a phylogeny and trait table*

Description

Matches a phylogeny and trait table without silently discarding information. The returned object contains the matched tree, the trait table reordered in tree-tip order, and a report listing matched species, removed tree tips, and removed data rows.

Usage

```
match_tree_data(tree, data = NULL, prune = TRUE, verbose = TRUE, X = NULL)
```

Arguments

tree	An object of class "phylo" or a prepared tree context.
data	Named vector, matrix, or data.frame. Species names must be supplied as names or row names unless rows are already in tree$tip.label order.
X	Deprecated compatibility alias for data. Supply only one of data and X.
prune	Whether to drop tree tips that are absent from X.
verbose	Whether to print a short matching message.

Value

A list with elements tree, X, data, report, matched_species, tree_tips_removed, and data_rows_removed. data is an alias of X for the recommended check-resolve-match-calculate workflow.

Examples

```
set.seed(6)
tree <- ape::rtree(10)
X <- data.frame(trait = stats::rnorm(10), row.names = tree$tip.label)
matched <- match_tree_data(tree, data = X, verbose = FALSE)
matched$report
```

plot_signal *Plot fastphylosig results*

Description

Creates lightweight base R diagnostic plots for supported fastphylosig result objects. For K and Delta, the x axis is the final phylogenetic-signal value and the density/ridge shows simulated signal values from the permutation test; the right-tail region `simulated >= observed` is shaded to show the P-value tail. For single-trait D results, `null = "auto"` draws a Fritz & Purvis D calibration plot: Brownian threshold and random association null distributions are scaled onto the D axis and shown together as overlaid mountain densities on the same baseline, the observed D is drawn as a vertical line, and reference lines mark D = 0 (Brownian expectation) and D = 1 (random expectation). The internal simulated contrast sums are converted back to D values before plotting. `P_random` and `P_Brownian` are labelled separately; zero extreme counts are displayed as `P < 1/nsim` rather than `P = 0`. For lambda, which uses a deterministic likelihood-ratio test rather than random simulations, the helper draws the stored profile log-likelihood curve when available, marks `lambda_hat`, `lambda = 0`, `lambda = 1`, and an approximate 95 percent profile-likelihood confidence interval. The confidence interval is the region with log-likelihood at least `max(logL) - 0.5 * qchisq(0.95, df = 1)`. Old lambda results without a stored profile fall back to a compact likelihood-ratio summary. Observed signal values, P values, and simulation counts are printed on the plots where applicable.

Usage

```
plot_signal(result, p_col = NULL, alpha = 0.05, main = NULL,
  xlab = NULL, ylab = NULL,
  null = c("auto", "random", "brownian"),
  label = c("none", "significant", "top", "all"), top_n = 10,
  ridge_scale = 0.85, col = NULL, pch = 19, cex = 0.7, ...)
```

Arguments

`result`	Result produced by `fast_signal`, `fast_k`, `fast_lambda`, `fast_d`, or `fast_delta`.
`p_col`	Optional P-value column for data.frame results. For D results the default selects the fitted random-null probability. Recognized canonical and legacy aliases select their corresponding null under `null = "auto"`. An explicitly present alias is honored even if another alias differs. A custom table column is retained under `null = "auto"`; an explicit `null` takes precedence over a P-value column for the other null.
`alpha`	Significance level used for coloring observed signal lines.
`null`	For D calibration plots, which null simulation to plot. `"auto"` draws both the random and Brownian D null distributions for a single-trait `fast_d()` result.
`label`	Which observed traits to label in multi-trait ridge plots.
`top_n`	Number of smallest-P traits to label when `label = "top"`.
`ridge_scale`	Vertical scaling of each density ridge in multi-trait plots.
`main, xlab, ylab, col, pch, cex, ...`	Base plotting arguments.

Value

Invisibly returns the normalized plotting data. K and Delta plots include sim, null_model, n_sim, and tail_direction. Single-trait D overlay plots include sim_random, sim_brownian, Pval1, Pval0, n_sim, plot_type = "D_calibration", P_random_display, and P_Brownian_display. Lambda plots include plot_type = "lambda_profile" when a stored profile is available, plus profile, ci_lower, ci_upper, and LR. Old lambda results without a profile return plot_type = "lambda_lr_summary".

Examples

```
set.seed(7)
tree <- ape::rtree(20)
x <- stats::rnorm(20)
names(x) <- tree$tip.label
k <- fast_signal(tree, data = x, method = "K", test = TRUE, nsim = 20,
                 return_sim = TRUE, verbose = FALSE)
plot_signal(k)
```

---

prepare_tree *Prepare a phylogenetic tree for repeated signal calculations*

Description

Builds a reusable tree context containing linear-size topology and traversal data. Dense covariance matrices, Cholesky factors, lambda spectral data, and trait-specific numerical subsets are cached lazily when first requested. Every external reuse validates the V2 context schema and exact protected snapshot before cached or numerical work.

Usage

```
prepare_tree(tree, max_cached_subsets = 16L,
  cache_budget = 512 * 1024^2)
```

Arguments

tree	A rooted object of class "phylo" with finite, non-negative branch lengths.
max_cached_subsets	Maximum number of full or NA-pruned subset entries retained in the structural cache, including the full-tree entry. Retained for compatibility; dense memory is controlled separately by cache_budget.
cache_budget	Maximum number of bytes retained in the numerical LRU cache. A value of zero permits transient numerical calculations without retaining their dense payloads.

Details

The tree context does not alter the tree or raw trait data. Initial preparation stores a snapshot; later changes to the original phylo object do not update it. On reuse, a versioned exact protected snapshot covers the cached tree's tip labels, edge matrix, branch lengths, and Nnode. Changes to those fields inside the context are rejected before cache or numerical work, while node.label and other non-computational attributes are outside the integrity contract; node.label may be reordered when internal nodes are renumbered. V2 contexts can be saved with saveRDS() and reused after readRDS(); missing, legacy, or unknown context schemas must be rebuilt with prepare_tree(). Initial preparation stores only topology, tip mappings, and traversal data. Public production K and lambda use the tree-pruning engines and do not request dense covariance, Cholesky, or spectral payloads; D, Delta, and ACE do not request them either. The bounded numerical cache remains available to internal validation paths. Results retain the statistical formulas of the corresponding reference functions. See cache_info for cache diagnostics.

Value

An object of class c("fastphylosig_tree", "list"). Pass it as the tree argument to fast_signal(), fast_k(), fast_lambda(), fast_d(), and fast_delta(). For ACE, use fast_ace(x, prepared = ctx). The unified fast_signal() entry point still requires an explicit method.

Examples

```
set.seed(1)
tree <- ape::rtree(20)
ctx <- prepare_tree(tree)
X <- matrix(stats::rnorm(40), nrow = 20,
            dimnames = list(tree$tip.label, c("trait_1", "trait_2")))
fast_signal(ctx, data = X, method = "K", verbose = FALSE)
```

---

resolve_tree *Resolve a phylogenetic tree conservatively*

Description

Checks a tree with check_tree() and makes only safe representation normalizations (edge ordering and internal node numbering) under Canonicalization Contract V2. Tip IDs and labels remain unchanged; the root is n_tip + 1, and remaining ordering is derived from exact UTF-8 tip-label bytes rather than locale collation, source internal-node IDs, or source edge-row order. An unrooted tree is rooted only when the selected signal requires it and an explicit valid outgroup is supplied. The function does not midpoint-root, infer an outgroup, resolve polytomies, collapse unary nodes, modify branch lengths, jitter zero branches, or drop tips. Remaining blocking conditions produce an explicit USER_ACTION_REQUIRED error.

Usage

```
resolve_tree(tree, signal, outgroup = NULL)
```

Arguments

tree	An object of class "phylo".
signal	One of "K", "lambda", "D", or "Delta"; matching is case-insensitive and exactly one value is required.
outgroup	Optional character vector of one or more existing tip labels. It is used only when the selected signal requires a rooted tree.

Details

After every representation or rooting change the tree is checked again. A successful result is idempotent: resolving it again does not change its topology, tip labels, or branch-length paths.

Value

A "phylo" object with class "fastphylosig_resolved_tree" and an attribute named "fastphylosig_resolution". The attribute records the signal, original and final readiness statuses, representation changes, and a logical ready flag.

See Also

check_tree, prepare_tree

Examples

```
tree <- ape::rtree(8)
tree <- resolve_tree(tree, signal = "K")
attr(tree, "fastphylosig_resolution")
```

---

result_methods *Stable result classes and display methods*

Description

The public calculation functions retain their historical fields and legacy classes while adding a common result class. Scalar results inherit from fastphylosig_result after a statistic-specific class (fastphylosig_signal, fastphylosig_d, or fastphylosig_delta) and before the corresponding legacy class. Matrix and data-frame results inherit from c("fastphylosig_table", "fastphylosig_result", "data.frame").

Usage

```
## S3 method for class 'fastphylosig_result'
print(x, ..., digits = getOption("digits"))
## S3 method for class 'fastphylosig_result'
summary(object, ...)
## S3 method for class 'fastphylosig_result'
as.data.frame(x, row.names = NULL,
  optional = FALSE, ...)
```

Arguments

x	A scalar or table result returned by one of the fastphylosig calculation functions.
object	A result supplied to summary().
row.names	Optional row names for the data-frame representation.
optional	Logical flag accepted for compatibility with the base as.data.frame() generic.
digits	Number of significant digits used by print().
...	Additional arguments passed to the underlying data-frame printer.

Details

Canonical aliases are additive. K results expose estimate, p_value, p_mcse, n_sim_requested, and n_sim_successful; lambda results expose estimate, logLik, LR, and p_value. D results keep the random and Brownian P values and Monte Carlo errors separate (P_random, P_Brownian, MCSE_P_random, and MCSE_P_Brownian); no combined p_value is created. Delta results expose estimate, p_value, p_mcse, estimate_mcse, chain-specific ESS/R-hat aliases, and simulation or iteration count aliases.

The no-test K scalar remains an atomic numeric with its legacy phylosig class. The display methods derive its estimate directly and do not coerce it to a list. Printing, summarising, and coercing results to a data frame use a compact canonical schema and omit retained simulation vectors.

Value

print() returns the input invisibly. summary() and as.data.frame() return a compact data frame; the latter has one row for a scalar result and preserves all non-simulation columns for a result table.

See Also

fast_signal, fast_k, fast_lambda, fast_d, fast_delta

Index

as.data.frame.fastphylosig_result
(result_methods), 19
as.data.frame.fastphylosig_table
(result_methods), 19

cache_info, *2*, 3, *18*
check_tree, *2*, 4

fast_ace, *2*, 5
fast_d, *2*, 6, *16*
fast_delta, *2*, 8, *16*
fast_k, *2*, 10, *16*
fast_lambda, *2*, 11, *16*
fast_signal, *2*, 13, *16*
fastphylosig (fastphylosig-package), 2
fastphylosig-package, 2

match_phylo_data, 14
match_tree_data, *2*, *15*, 15

plot_signal, *2*, 16
prepare_tree, *2*, 17
print.fastphylosig_result
(result_methods), 19
print.fastphylosig_table
(result_methods), 19

resolve_tree, *2*, 18
result_methods, 19

summary.fastphylosig_result
(result_methods), 19
summary.fastphylosig_table
(result_methods), 19